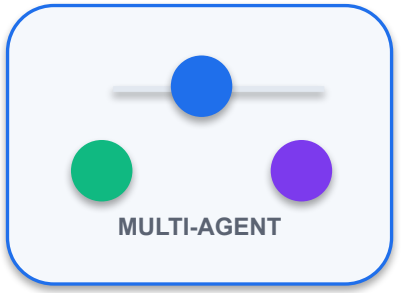



COURSE SLIDES · WSQ

# AI Vibe Coding for Multi-Agents System

WSQ Course Code: TGS-2020503207

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr Alfred Ang

Version v1.0 · 11 August 2026

COURSE ADMINISTRATION

# Welcome & Housekeeping

# Digital Attendance (Mandatory)

1. The trainer displays the SSG digital attendance QR code
2. Scan the QR code with your mobile phone camera
3. Submit your attendance — AM, PM and Assessment
4. Repeat on every training day
5. 75% attendance is required for assessment and funding

# About the Trainer


## Your Trainer

General Trainer template —
to be completed by the trainer

**NAME**

**TITLE / DESIGNATION**

**QUALIFICATIONS**

**AREAS OF EXPERTISE**

**TRAINING & INDUSTRY EXPERIENCE**

**CONTACT**

YOUR TRAINER

# About the Trainer


## Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

**ROLE**
Principal Trainer, Tertiary Infotech Academy Pte Ltd

**EXPERTISE**
AI agents, multi-agent systems, machine learning and full-stack AI application development.

**DELIVERS**
WSQ courses on AI vibe coding, agentic AI, machine learning and data analytics.

**PROFILE**
Founder and lead instructor at Tertiary Infotech / Tertiary Courses · github.com/alfredang

ICE-BREAKER

# Let's Know Each Other

- Your name, organisation and role.
- Your experience with Python, LLMs or AI coding tools (if any).
- What you want to be able to build with AI agents after this course.

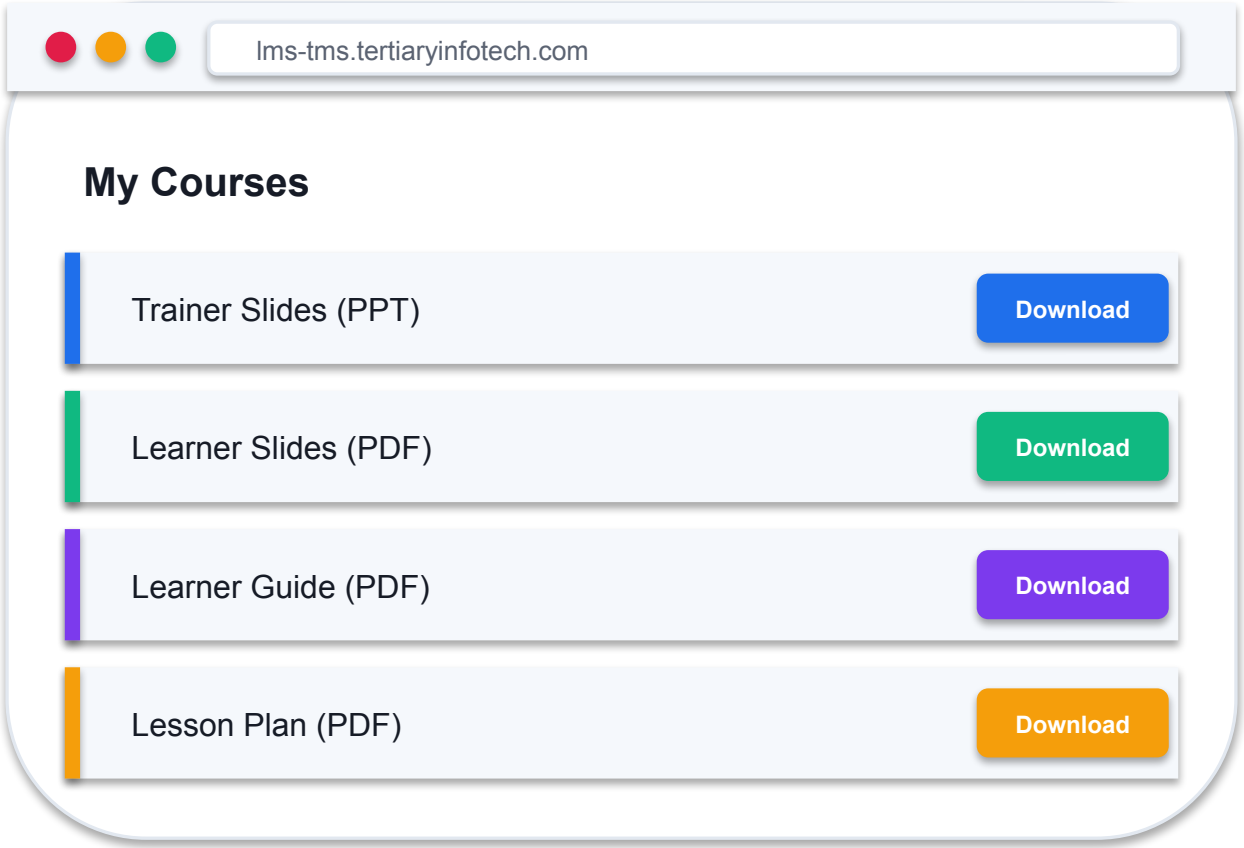
HOUSEKEEPING

# Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required for funding.

COURSE PORTAL · LMS/TMS

# Download Your Course Material




1. Go to lms-tms.tertiaryinfotech.com and sign in with the email you registered with.
2. Open My Courses and select this course.
3. Download the Trainer Slides, Learner Guide and Lesson Plan.
4. Keep them open — the assessment is OPEN BOOK.

TSC MAPPING

# Skills Framework Alignment

1

## Analytics and Computational Modelling

TSC Code: ICT-DIT-3001-1.1

2

## Ability A1

Identify appropriate statistical algorithms and data models to test hypotheses or theories

3

## Ability A2

Use appropriate analytics platforms and analytical tools given specific analytics and reporting requirements

4

## Ability A3

Utilise a range of statistical methods and analytics approaches to analyse data

5

## Ability A4

Evaluate the performance and suitability of models against business requirements

SCHEDULE

# Lesson Plan — 2 Days, 8 Hours per Day

## Day 1

- Day 1 — Agent Foundations & Vibe Coding for Multi-Agent Systems
  - Digital Attendance (AM), introductions, learning outcomes
  - Topic 1: Modern Agent Foundations (Labs 1–2)
  - Topic 2: Vibe Coding for Multi-Agent Systems (Labs 3–4)
  - Topic 3 begins: OpenAI Agents SDK (Lab 5)
  - Day 1 recap and PM digital attendance

## Day 2 & timing

- Day 2 — Agent SDKs, MCP, Sub-Agents & Assessment
  - Topic 3 continues: supervisor routing, Streamlit (Labs 6–7)
  - Topic 4: Gemini Agent SDK (Labs 8–9)
  - Topic 5: MCP and Sub-Agents (Labs 10–11)
  - Course feedback and TRAQOM survey
  - Final Assessment: WA (SAQ) + PP
  - Daily timing: 9:30am–7:00pm · 8 instructional hours · 1-hour lunch · 2 tea breaks

WHAT YOU'LL ACHIEVE

# Learning Outcomes

1

## LO1 · Agent Foundations

Skills, memory, tools, MCP, and single-agent to multi-agent progression.

2

## LO2 · Vibe Coding

A structured workflow with context engineering for reliable agent code.

3

## LO3 · OpenAI Agents SDK

Structured outputs, tool calling, supervisor routing, Streamlit deployment.

4

## LO4 · Gemini Agent SDK

Collaborative Gemini agents, deployed with Streamlit.

5

## LO5 · MCP & Sub-Agents

Tool orchestration and hierarchical sub-agent architecture.

FIVE TOPICS · ELEVEN HANDS-ON LABS

# Course Outline

1

## Topic 01 — Modern Agent Foundations

Agent anatomy · Skills · Memory · MCP · Single-agent to multi-agent

2

## Topic 02 — Vibe Coding for Multi-Agent Systems

Vibe coding workflow · Context engineering · Tooling · Agent skills

3

## Topic 03 — Multi-Agent System Development with OpenAI Agents SDK

Agent design · Structured outputs · Tool calling · Supervisor routing · Streamlit

4

## Topic 04 — Multi-Agent System Development with Gemini Agent SDK

Gemini agent design · Collaborative agents · Streamlit deployment

5

## Topic 05 — MCP and Sub-Agents

MCP servers · Tool orchestration · Hierarchical sub-agent architecture

BEFORE THE ASSESSMENT

# Briefing for Assessment

1

## Phones away

Place phones and other materials under the table or on the floor.

2

## No recording

No photos or recording of assessment scripts.

3

## Work alone

No discussion of any kind during the assessment.

4

## Black or blue pen

Use a black/blue pen for hard-copy assessments.

5

## No correction fluid

No liquid paper or correction tape on the script.

6

## Time is called

Scripts are collected when time is up.

FINAL ASSESSMENT

# Assessment

## Written (WA)

- Short-Answer Questions (SAQ)
- 50 minutes
- Tests underpinning knowledge
- Answer in your own words

## Practical (PP)

- Hands-on multi-agent build tasks
- 75 minutes
- Tests practical ability
- Based on the labs you did in class

## Rules

- Open book — slides, Learner Guide and approved materials
- 75% attendance required
- Graded Competent / Not Yet Competent
- An appeal process is available

ON ASSESSMENT DAY

# Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

LAB RESOURCES

# Codes for the Labs

1

## GitHub repository

github.com/tertiarycourses/TGS-2020503207-AI-Vibe-Coding-for-Multi-Agents-System

2

## 11 lab files

One Markdown file per lab, with the complete runnable code.

3

## Clone or browse

Clone the repo, or open any lab file directly in your browser.

4

## Learner Guide

The full detailed step-by-step for every lab — your open-book reference.

CORE CONCEPTS

# From One Agent to Many

FOUNDATIONS

# What is an AI Agent?

1

## Not just a chatbot

A chatbot answers; an agent pursues a goal, takes actions and checks its own results.

2

## The reasoning loop

Reason, act, observe — repeated until the goal is met or a limit is reached.

3

## Tools

Functions the agent may call to reach beyond the model: APIs, files, databases, the shell.

4

## Memory

What it carries forward — the conversation now, and durable facts across sessions.

5

## Autonomy with limits

The agent chooses the path; you set the boundaries, the budget and the review gate.

6

## Why now

Reliable tool calling and long context finally make the loop dependable enough to ship.

HOW AN AGENT ACTUALLY WORKS

# The Agent Reasoning Loop

WHEN TO SPLIT

# Single Agent vs Multi-Agent System

## SINGLE AGENT

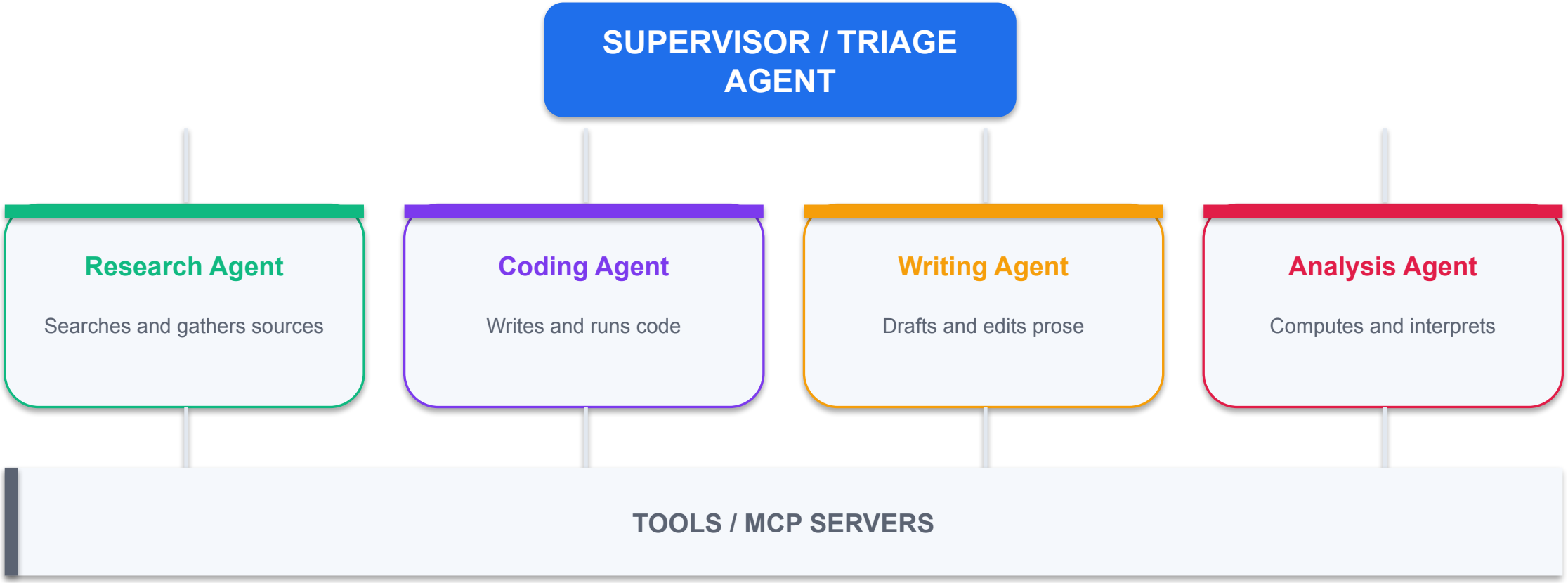
- Best when
  - The task is narrow and well defined
  - A handful of tools is enough
  - Context stays small
- Limits
  - Too many tools confuse tool choice
  - One long context mixes unrelated work
  - One instruction set must cover everything

## MULTI-AGENT SYSTEM

- Best when
  - The work splits into distinct specialities
  - Each part needs its own tools and rules
  - Parts can run in parallel
- Benefits
  - Focused instructions per agent
  - Isolated context per agent
  - Easier to test and replace one part

THE PATTERN YOU WILL BUILD TODAY

# Multi-Agent Architecture Pattern



WHY MULTI-AGENT

# One agent that does everything does nothing well.

Split the work by speciality, give each agent its own tools and context, and let a supervisor route the request.

THE UNIVERSAL TOOL INTERFACE

# Model Context Protocol (MCP)

1

## The problem

Every framework needed its own bespoke integration for the same tool.

2

## The standard

MCP is an open protocol for exposing tools, resources and prompts to any client.

3

## Servers

A server publishes typed tools — you write the capability once.

4

## Clients

Any MCP-aware agent discovers those tools and calls them.

5

## Transport

stdio for local servers; HTTP/SSE for remote ones.

6

## The payoff

Write it once, and every MCP-aware agent can use it.

01

TOPIC 01

# Modern Agent Foundations

Agent anatomy · Skills · Memory · MCP · Single-agent to multi-agent

TOPIC 01 · 20% OF THE COURSE

# Key Concepts — Modern Agent Foundations

1

## What is a modern AI agent?

An LLM that reasons in a loop, calls tools, keeps memory and pursues a goal — not a single prompt-and-response.

2

## Agent skills

Modular, transferable capabilities packaged so an agent can load only what a task needs, and reuse them across projects.

3

## Agent memory

Short-term context versus long-term stores; what to remember, what to summarise and what to discard.

4

## Model Context Protocol (MCP)

An open standard that lets an agent discover and call external tools and data sources through a uniform interface.

5

## Tools and tool calling

The agent chooses a function, the runtime executes it, and the result re-enters the reasoning loop.

6

## Single-agent to multi-agent

When one agent's context and responsibilities grow too large, split the work across specialised collaborating agents.

WHAT YOU'LL DO

# Hands-On Labs — Modern Agent Foundations

## Lab 1

- Build Your First Tool-Calling Agent

## Lab 2

- Agent Memory and Reusable Agent Skills

## —

- —

TOPIC 01 · HANDS-ON

# Build Your First Tool-Calling Agent

**LAB 1**

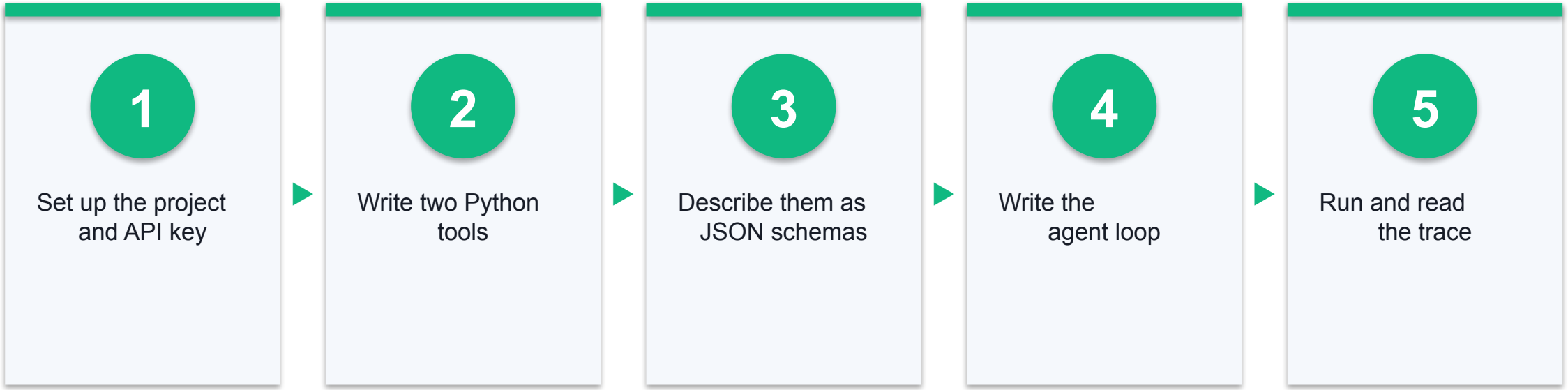
Build a single agent from first principles so the loop is not a black box: the model receives a goal, decides to call a tool, your code executes it, and the result is fed back until the agent can answer. You will see every message that crosses the boundary.

**You'll build**

**A runnable Python agent that answers questions by calling two of your own tools, printing each reasoning step and tool result so the loop is visible.**

**Tools:** Python 3.11+, OpenAI Python SDK, uv/pip, .env for API keys

# Lab 1 Workflow



LAB 1 · VERIFY

# Build Your First Tool-Calling Agent

## ✅ Test it

The agent answers both parts of the question in one run, and the printed trace shows it called get_weather once and calculate once before producing the final answer.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 01 · HANDS-ON

# Agent Memory and Reusable Agent Skills

**LAB 2**

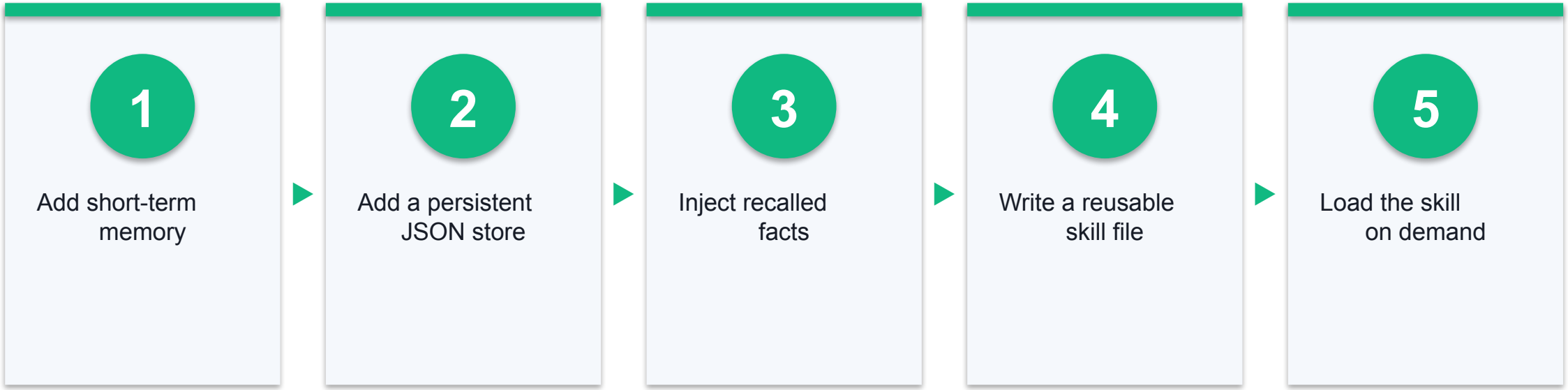
Give the agent memory so it stops forgetting between turns, then extract a repeatable procedure into a skill file the agent loads on demand — the modular, transferable capability described in the course outline.

**You'll build**

**An agent with a conversation buffer plus a persistent JSON memory store, and one reusable skill file that the agent loads only when the task calls for it.**

**Tools:** Python, OpenAI SDK, JSON file store, Markdown skill definition

# Lab 2 Workflow



LAB 2 · VERIFY

# Agent Memory and Reusable Agent Skills

## ✅ Test it

Tell the agent a fact, restart the process, ask about it again — the agent recalls it from memory.json. With the skill loaded, the agent follows the documented procedure instead of improvising.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC RECAP

# Recap — Modern Agent Foundations

- You can now: Explain the components of a modern AI agent and implement a reasoning loop with tool calling
- You can now: Implement short-term and long-term agent memory, and package a capability as a reusable skill

02

TOPIC 02

# Vibe Coding for Multi-Agent Systems

Vibe coding workflow · Context engineering · Tooling · Agent skills

TOPIC 02 · 20% OF THE COURSE

# Key Concepts — Vibe Coding for Multi-Agent Systems

1

### What is vibe coding?

Directing an AI coding agent in natural language to generate, run and refine working software, while you stay the reviewer and architect.

2

### The structured workflow

Specify, then scaffold, then generate, then run, then verify, then refine — a disciplined loop, not one-shot prompting.

3

### Context engineering

Curating exactly the files, specs and constraints the agent sees, which drives reliability far more than prompt wording.

4

### Vibe coding tools

Claude Code, Gemini CLI, Codex CLI, Cursor, Windsurf, Antigravity and Trae — terminal agents and agentic IDEs.

5

### Agent skills integration

Packaging repeatable procedures as skills the coding agent invokes, so quality does not depend on re-explaining each time.

6

### Human in the loop

You own the specification, the review and the acceptance test; the agent owns the typing.

SPECIFY → SCAFFOLD → GENERATE → VERIFY → REFINE

# The Structured Vibe Coding Workflow

1. SPECIFY — write the spec and the acceptance test first
2. SCAFFOLD — let the agent build the structure
3. GENERATE — implement one component at a time
4. RUN & VERIFY — execute, paste real errors back
5. REFINE — one improvement per iteration, commit each

CHOOSE YOUR AGENT

# Vibe Coding Tools

## TERMINAL AGENTS

### Claude Code

claude.com/product/claude-code

### Gemini CLI

geminicli.com/

### Codex CLI

developers.openai.com/codex/cli/

### Grok CLI

grokcli.io/

## AGENTIC IDEs

### Cursor

cursor.com/

### Windsurf

windsurf.com/

### Google Antigravity

antigravity.google/

### Trae

www.trae.ai/

THE HIGHEST-LEVERAGE SKILL

# Context Engineering — What Actually Drives Reliability

1

## Curate, don't dump

Show the agent the few files that matter, not the whole repository.

2

## Project context file

CLAUDE.md / GEMINI.md states the stack, conventions and prohibitions once.

3

## Precise references

Point at file paths and line ranges instead of pasting large blocks.

4

## Skills

Package a repeatable procedure so quality does not depend on re-explaining it.

5

## Specify before generating

A written spec and acceptance test beat any amount of prompt wording.

6

## Stay the reviewer

Read every diff. You own the architecture and the acceptance decision.

WHAT YOU'LL DO

# Hands-On Labs — Vibe Coding for Multi-Agent Systems

## Lab 3

- Vibe Code an Agent with a Structured Workflow

## Lab 4

- Context Engineering and a Reusable Coding Skill

## —

- —

TOPIC 02 · HANDS-ON

# Vibe Code an Agent with a Structured Workflow

**LAB 3**

Use an AI coding agent to build a working application without hand-writing the implementation, following a disciplined loop rather than one-shot prompting. The specification you write is the real deliverable; the agent produces the code.

**You'll build**

**A working research-assistant agent generated by a coding agent from your written specification, with an acceptance test you defined before any code existed.**

**Tools:** Claude Code / Gemini CLI / Cursor, Python, Git

# Lab 3 Workflow

1. Write SPEC.md and the test
2. Scaffold the structure
3. Generate one component at a time
4. Run and paste real errors back
5. Verify, commit, refine

LAB 3 · VERIFY

# Vibe Code an Agent with a Structured Workflow

## ✅ Test it

The generated agent passes the acceptance test you wrote in SPEC.md before any code existed, and the Git history shows small reviewable commits rather than one giant unreviewed dump.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 02 · HANDS-ON

# Context Engineering and a Reusable Coding Skill

**LAB 4**

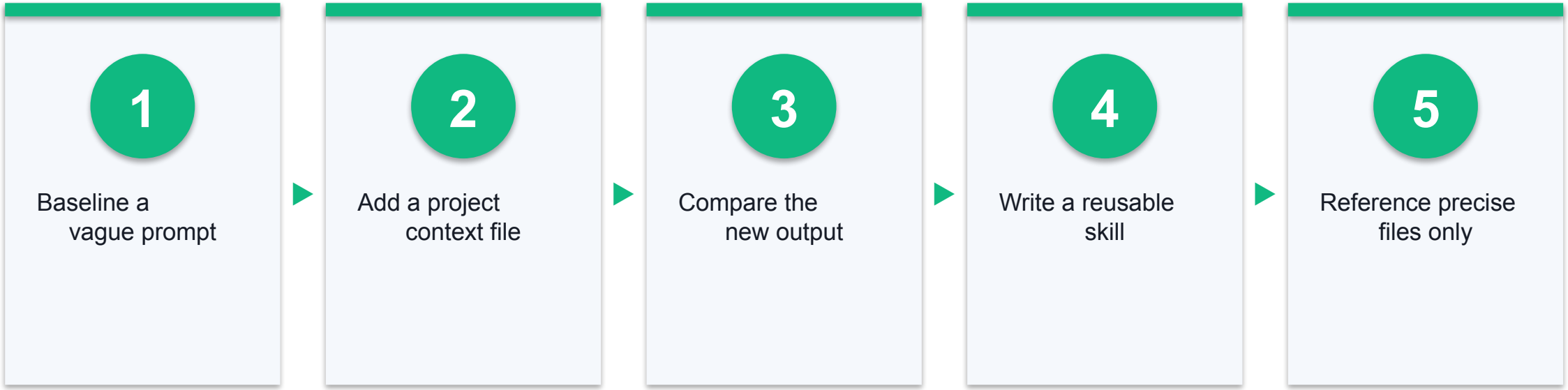
Improve reliability by controlling exactly what the coding agent sees, then capture your house conventions as a skill so the agent applies them automatically instead of you re-explaining them in every session.

**You'll build**

**A project configured with a context file and a reusable skill, demonstrably producing house-standard code without repeated instructions.**

**Tools:** Claude Code / Gemini CLI, Markdown, Git

# Lab 4 Workflow



LAB 4 · VERIFY

# Context Engineering and a Reusable Coding Skill

## ✅ Test it

With the context file and skill in place, the same prompt that previously produced inconsistent code now yields code matching your stated conventions, without you restating them.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC RECAP

# Recap — Vibe Coding for Multi-Agent Systems

- You can now: Apply the structured vibe coding workflow — specify, scaffold, generate, run, verify, refine
- You can now: Engineer the agent's context and package a repeatable procedure as a project skill

03

TOPIC 03

# Multi-Agent System Development with OpenAI Agents SDK

Agent design · Structured outputs · Tool calling · Supervisor routing · Streamlit

TOPIC 03 · 25% OF THE COURSE

# Key Concepts — Multi-Agent System Development with OpenAI Agents SDK

1

### The OpenAI Agents SDK

A lightweight Python framework for agents, tools, handoffs and guardrails, with tracing built in.

2

### Agents and instructions

An agent is a model plus instructions plus a tool set; clear role boundaries make multi-agent behaviour predictable.

3

### Structured outputs

Pydantic output types force the model to return validated, typed data your code can rely on.

4

### Function tools and API calls

Decorate a Python function as a tool so the agent can call your own logic and external APIs.

5

### Supervisor routing and handoffs

A triage agent classifies the request and delegates to the specialist sub-agent best suited to it.

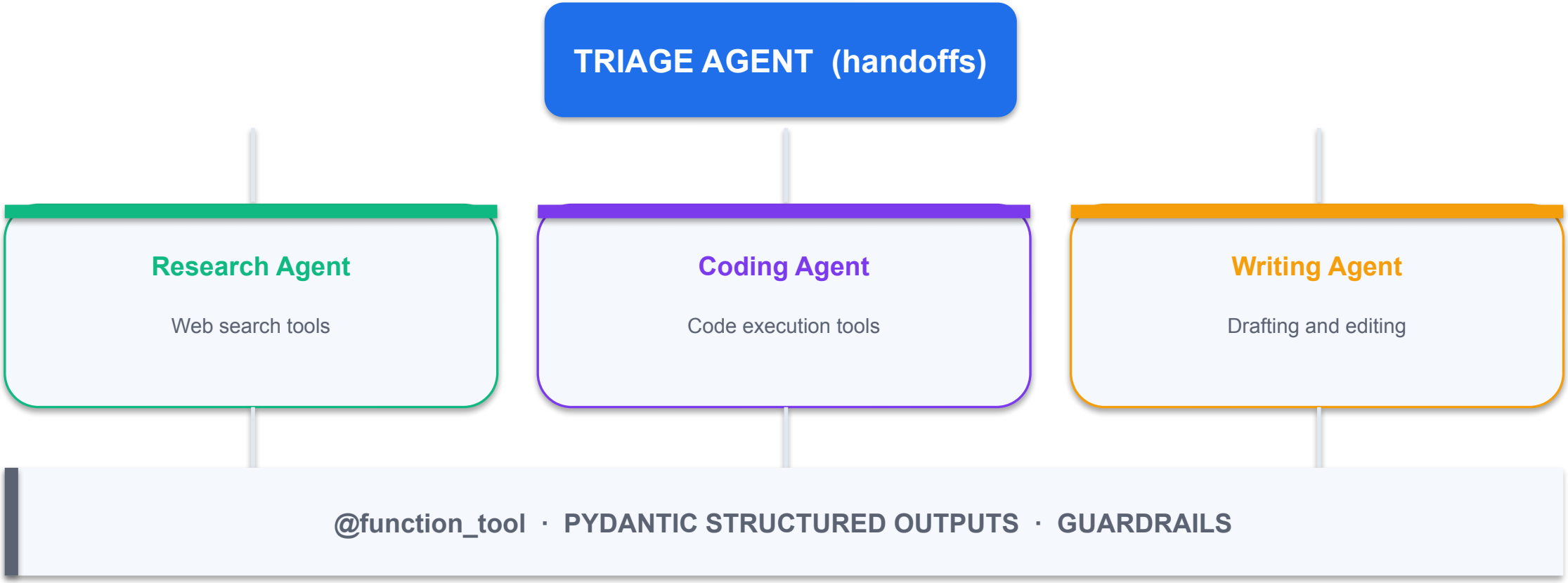
6

### Streamlit deployment

Wrap the multi-agent system in a simple web UI so non-developers can use it.

TOPIC 03 · ARCHITECTURE

# OpenAI Agents SDK — Triage and Handoffs



OPENAI AGENTS SDK

# The Building Blocks

1

## Agent

A model plus instructions plus tools — the unit of specialisation.

2

## Runner

Executes the agent loop: Runner.run_sync(agent, prompt).

3

## @function_tool

Turns a Python function into a callable tool from its type hints and docstring.

4

## output_type

A Pydantic model that forces validated, typed output instead of free prose.

5

## handoffs

The list of specialists a triage agent may delegate to.

6

## Guardrails

Input and output checks that stop out-of-scope or unsafe work early.

WHAT YOU'LL DO

# Hands-On Labs — Multi-Agent System Development with OpenAI Agents SDK

## Lab 5

- First Agent with the OpenAI Agents SDK

## Lab 6

- Supervisor Routing and Sub-Agent Delegation

## Lab 7

- Deploy the Multi-Agent System with Streamlit

TOPIC 03 · HANDS-ON

# First Agent with the OpenAI Agents SDK

**LAB 5**

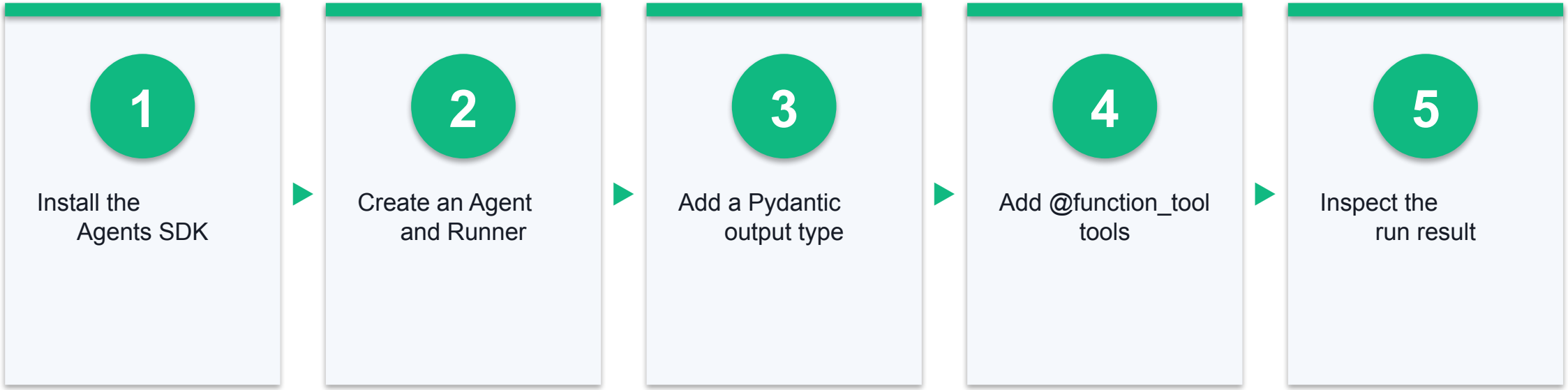
Rebuild the hand-rolled agent from Lab 1 on the OpenAI Agents SDK and see how much the framework removes. Add a Pydantic output type so the agent returns validated typed data instead of prose, and expose your own Python functions as tools.

**You'll build**

**An SDK-based agent that calls your function tools and returns a validated Pydantic object your code can use directly.**

**Tools:** Python 3.11+, openai-agents SDK, Pydantic, .env

# Lab 5 Workflow



LAB 5 · VERIFY

# First Agent with the OpenAI Agents SDK

## ✅ Test it

result.final_output is an instance of your Pydantic model with correctly typed fields, and the run trace shows the expected tool calls.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 03 · HANDS-ON

# Supervisor Routing and Sub-Agent Delegation

**LAB 6**

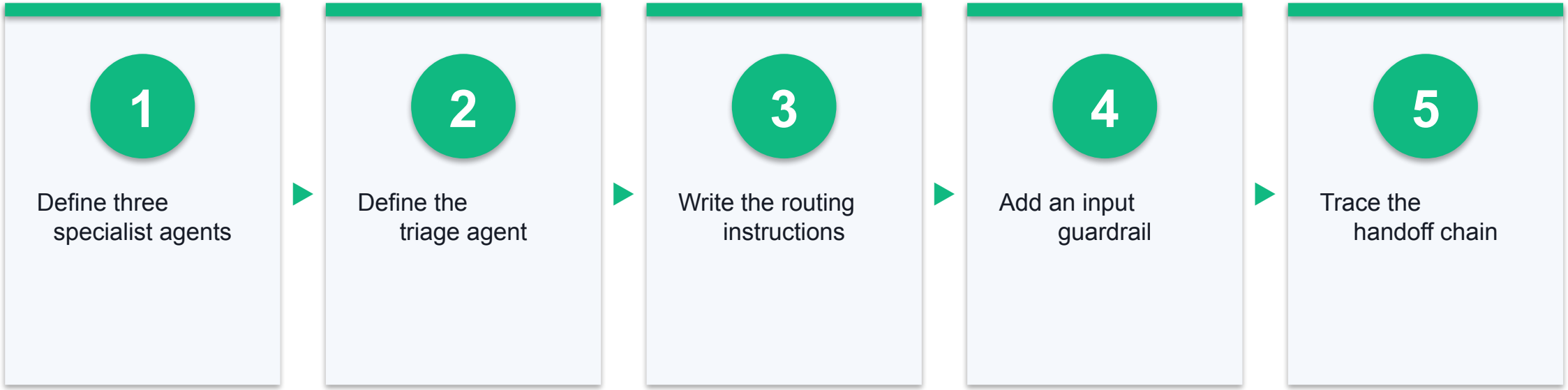
Move from one overloaded agent to a team. Build three specialists and a triage supervisor that classifies each request and hands it to the right one — the core multi-agent pattern of the course.

**You'll build**

**A four-agent system — a triage supervisor plus research, coding and writing specialists — that routes each request to the correct specialist.**

**Tools:** Python, openai-agents SDK, Pydantic

# Lab 6 Workflow



LAB 6 · VERIFY

# Supervisor Routing and Sub-Agent Delegation

## ✅ Test it

A research question reaches the research agent, a coding request reaches the coding agent, and an out-of-scope request is refused by the guardrail before any specialist runs.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 03 · HANDS-ON

# Deploy the Multi-Agent System with Streamlit

**LAB 7**

Put a web interface on the agent team from Lab 6 so a non-developer can use it, streaming the reply and showing which agent handled each request.

**You'll build**

**A running Streamlit chat application backed by your multi-agent system, with visible routing and persistent conversation history.**

**Tools:** Python, Streamlit, openai-agents SDK

# Lab 7 Workflow

1. Install Streamlit and create app.py
2. Build the chat UI
3. Hold history in session_state
4. Show the routing to the user
5. Stream the response

LAB 7 · VERIFY

# Deploy the Multi-Agent System with Streamlit

## ✅ Test it

The app runs at localhost:8501, answers a multi-turn conversation with history intact, names the specialist that handled each turn, and contains no hard-coded API key.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC RECAP

# Recap — Multi-Agent System Development with OpenAI Agents SDK

- You can now: Design an agent with the OpenAI Agents SDK using structured outputs and function tools
- You can now: Construct a collaborative multi-agent system with supervisor routing and handoffs
- You can now: Deploy a collaborative multi-agent system as a shareable Streamlit web application

04

TOPIC 04

# Multi-Agent System Development with Gemini Agent SDK

Gemini agent design · Collaborative agents · Streamlit deployment

TOPIC 04 · 20% OF THE COURSE

# Key Concepts — Multi-Agent System Development with Gemini Agent SDK

1

## The Google Gemini Agent SDK

Google's Agent Development Kit (ADK) for building, composing and running Gemini-powered agents in Python.

2

## Defining a Gemini agent

Model, instruction, description and tools — the description is what makes an agent discoverable by its coordinator.

3

## Function tools in ADK

Plain Python functions become tools; the docstring and type hints tell the model how and when to call them.

4

## Multi-agent composition

Sub-agents attached to a coordinator agent, which routes each request to the right specialist.

5

## Sessions and runners

The runner executes the agent loop while the session service carries conversational state.

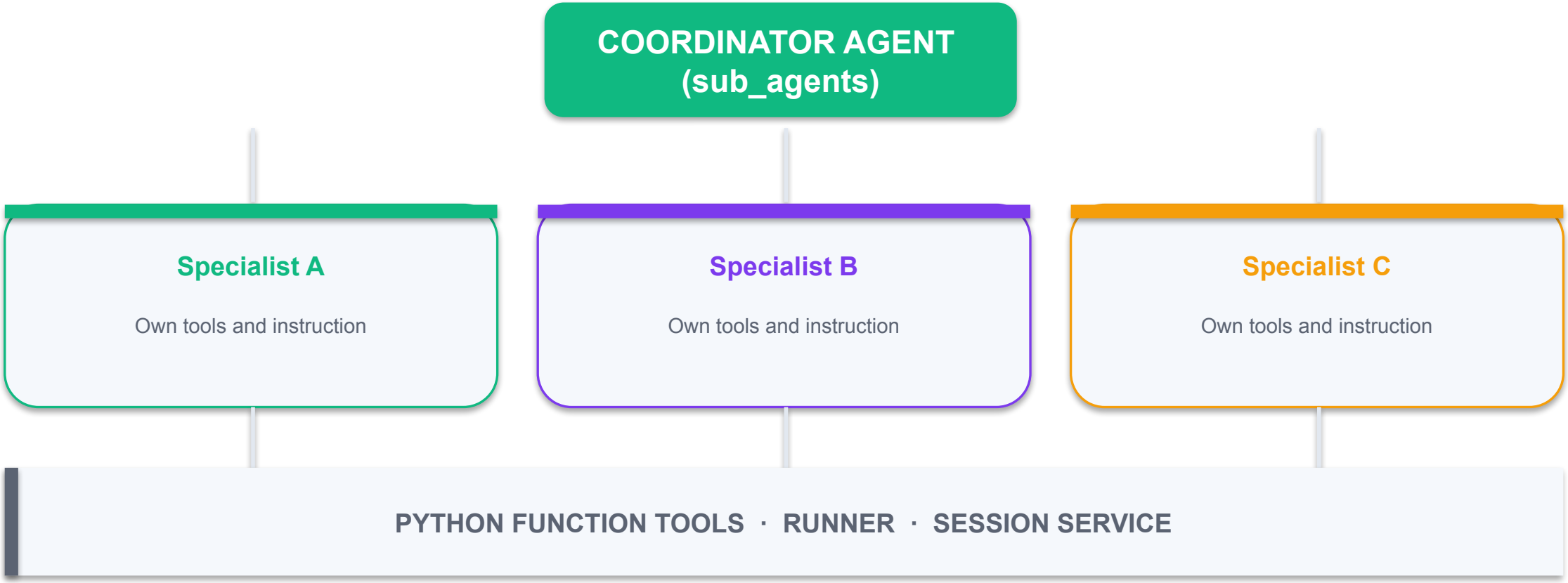
6

## Comparing the SDKs

The same multi-agent pattern expressed in two ecosystems — portable concepts, different APIs.

TOPIC 04 · ARCHITECTURE

# Gemini Agent SDK (ADK) — Coordinator and Sub-Agents



SAME PATTERN, TWO ECOSYSTEMS

# OpenAI Agents SDK vs Gemini ADK

## OPENAI AGENTS SDK

- Package: openai-agents
  - Agent(name, instructions, tools)
  - Runner.run_sync(agent, prompt)
  - Delegation via handoffs=[...]
  - Typed output via output_type=Model
  - Tool via @function_tool

## GOOGLE GEMINI ADK

- Package: google-adk
  - Agent(name, model, description, instruction)
  - Runner + InMemorySessionService
  - Delegation via sub_agents=[...]
  - Routing driven by each agent's description
  - Tool via a plain Python function

WHAT YOU'LL DO

# Hands-On Labs — Multi-Agent System Development with Gemini Agent SDK

## Lab 8

- Build a Multi-Agent System with the Gemini Agent SDK

## Lab 9

- Deploy the Gemini Multi-Agent System with Streamlit

## —

- —

TOPIC 04 · HANDS-ON

# Build a Multi-Agent System with the Gemini Agent SDK

**LAB 8**

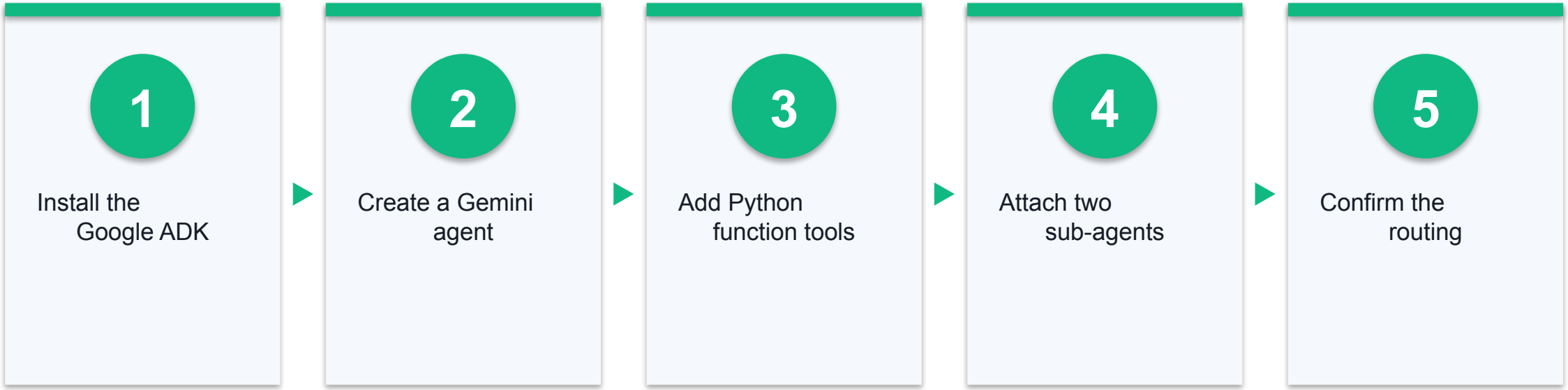
Express the same multi-agent pattern in Google's ecosystem. Building the equivalent system twice separates the portable concepts — agents, tools, routing — from one vendor's API surface.

**You'll build**

**A Gemini-powered coordinator agent with two specialist sub-agents and working function tools.**

**Tools:** Python 3.11+, google-adk, Google AI Studio API key

# Lab 8 Workflow



LAB 8 · VERIFY

# Build a Multi-Agent System with the Gemini Agent SDK

## ✅ Test it

The coordinator routes each request to the correct sub-agent, and the tools return live results — the same behaviour as the OpenAI build, in a different SDK.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 04 · HANDS-ON

# Deploy the Gemini Multi-Agent System with Streamlit

**LAB 9**

Give the Gemini agent team the same web interface treatment, then compare the two deployments side by side and record which ecosystem fits which kind of work.

**You'll build**

**A deployed Streamlit application backed by the Gemini multi-agent system, plus a short written comparison of the two SDKs.**

**Tools:** Python, Streamlit, google-adk

# Lab 9 Workflow

1. Import the ADK coordinator
2. Manage session and runner state
3. Render and stream the chat
4. Show the handling sub-agent
5. Compare the two SDKs

LAB 9 · VERIFY

# Deploy the Gemini Multi-Agent System with Streamlit

## ✅ Test it

The Gemini app runs, routes correctly and holds conversation state, and your README records a concrete comparison of the two SDKs.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC RECAP

# Recap — Multi-Agent System Development with Gemini Agent SDK

- You can now: Design collaborative agents with the Google Gemini Agent SDK (ADK)
- You can now: Deploy a Gemini-based collaborative multi-agent system with Streamlit

05

TOPIC 05

# MCP and Sub-Agents

MCP servers · Tool orchestration · Hierarchical sub-agent architecture

TOPIC 05 · 15% OF THE COURSE

# Key Concepts — MCP and Sub-Agents

1

## MCP in depth

Servers expose tools, resources and prompts; clients (your agent, Claude Code) discover and invoke them over a standard protocol.

2

## Why MCP matters

Write a capability once as a server, and every MCP-aware agent can use it — no bespoke integration per framework.

3

## Building an MCP server

Define typed tools with FastMCP and run them over stdio for local agents.

4

## Tool orchestration

Sequencing several tools to complete one goal, and handling partial failure sensibly.

5

## Hierarchical sub-agents

A parent delegates a bounded task to a child agent with its own context, tools and instructions.

6

## Context isolation

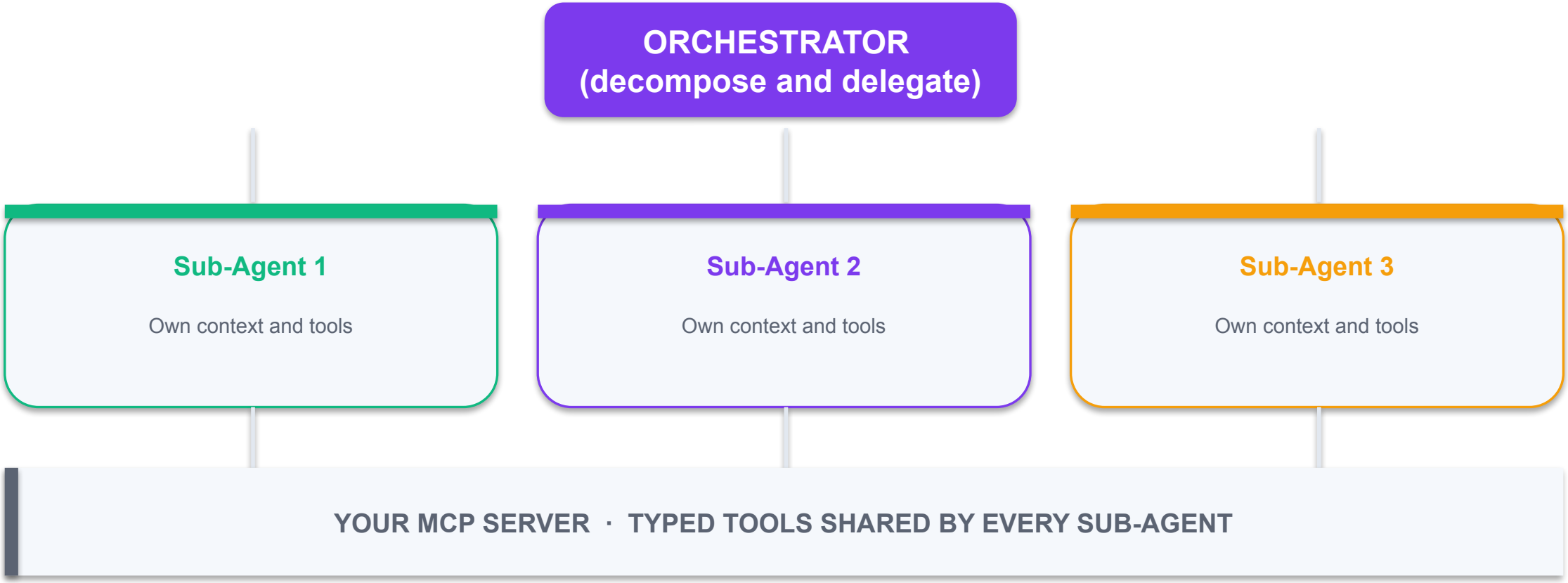
Sub-agents keep the parent's context clean by returning only the conclusion, not the whole working trace.

WRITE ONCE · USE FROM ANY MCP CLIENT

# How MCP Connects an Agent to Your Tools

1. You write an MCP server exposing typed tools
2. The server runs over stdio (or HTTP)
3. You register it with an MCP client
4. The client discovers the tool list
5. The agent calls tools as it reasons

# Hierarchical Sub-Agent Architecture



THE ARCHITECTURAL PAYOFF

# Why Context Isolation Matters

1

### The problem

One agent doing everything fills its context with irrelevant working detail.

2

### The fix

A sub-agent works in its own context and returns only the conclusion.

3

### The parent stays clean

The orchestrator sees answers, not transcripts.

4

### Parallelism

Independent sub-agents run concurrently, cutting wall-clock time.

5

### Failure containment

One failing sub-agent degrades the result instead of killing the run.

6

### Testability

Each sub-agent has a narrow contract you can test on its own.

WHAT YOU'LL DO

# Hands-On Labs — MCP and Sub-Agents

## Lab 10

- Build and Connect an MCP Server

## Lab 11

- Hierarchical Sub-Agents and Context Isolation

## —

- —

TOPIC 05 · HANDS-ON

# Build and Connect an MCP Server

**LAB 10**

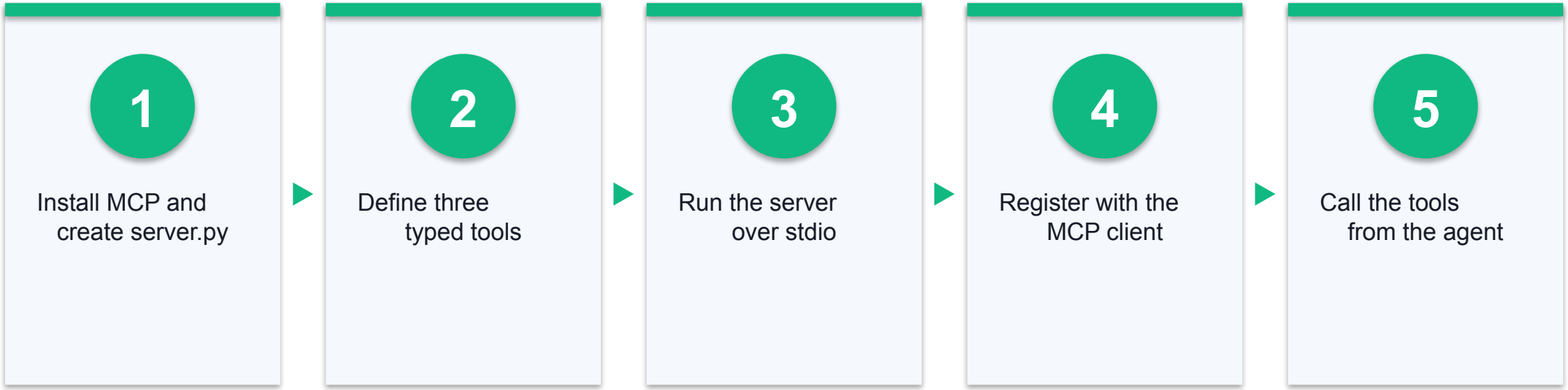
Write your own MCP server exposing typed tools, then connect it to a coding agent. Because MCP is an open standard, the same server works with any MCP-aware client — write the capability once, use it everywhere.

**You'll build**

**A working MCP server with three typed tools, connected to and callable from your coding agent.**

**Tools:** Python 3.11+, FastMCP (mcp package), Claude Code or another MCP client

# Lab 10 Workflow



LAB 10 · VERIFY

# Build and Connect an MCP Server

## ✅ Test it

The MCP client lists all three tools, and the agent completes a task that is impossible without them, calling the tools in a sensible order.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC 05 · HANDS-ON

# Hierarchical Sub-Agents and Context Isolation

LAB 11

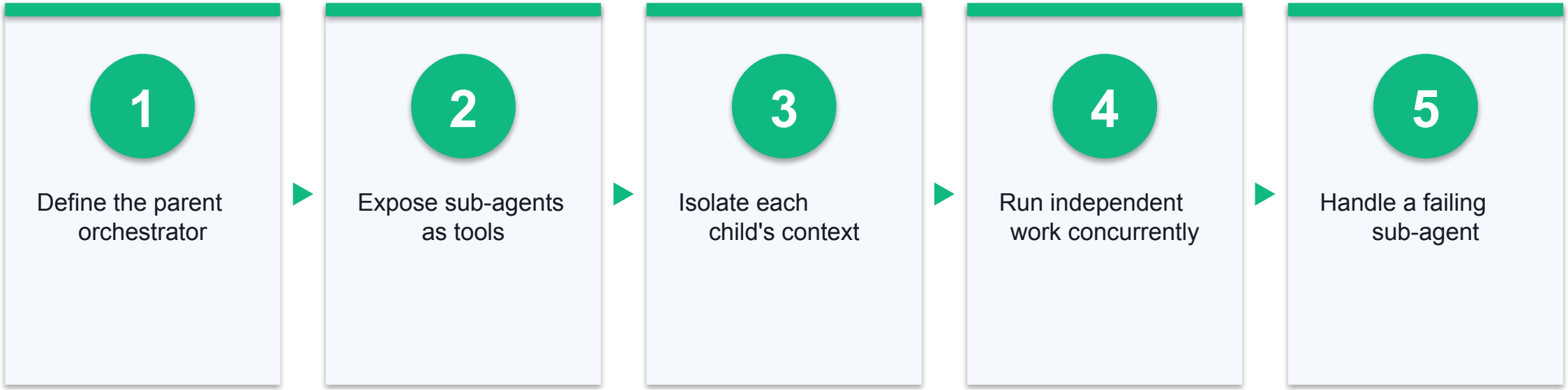
Build the top of the architecture: a parent agent that delegates bounded tasks to specialised child agents, each with its own context and tools. The children return conclusions, not their whole working trace, which is what keeps a large system reliable.

**You'll build**

**A three-level hierarchical system — orchestrator, sub-agents and MCP tools — that completes a multi-part task no single agent handles well.**

**Tools:** Python, openai-agents SDK or google-adk, your MCP server from Lab 10

# Lab 11 Workflow



LAB 11 · VERIFY

# Hierarchical Sub-Agents and Context Isolation

## ✅ Test it

The orchestrator completes the multi-part task by delegating to at least three sub-agents; the parent's context stays small because children return conclusions only, and one deliberately failing sub-agent does not crash the run.

Detailed step-by-step instructions for this lab are in the Learner Guide.

TOPIC RECAP

# Recap — MCP and Sub-Agents

- You can now: Orchestrate tools through the Model Context Protocol
- You can now: Design a hierarchical sub-agent architecture with delegated, isolated context

WRAP-UP

# Course Summary & Next Steps

LEARNING OUTCOMES

# What You Achieved

1

## LO1 · Agent Foundations

Built a tool-calling agent with memory and a reusable skill.

2

## LO2 · Vibe Coding

Applied specify-scaffold-generate-verify-refine with engineered context.

3

## LO3 · OpenAI Agents SDK

Built and deployed a supervisor-routed multi-agent system.

4

## LO4 · Gemini Agent SDK

Built and deployed the same pattern on Google's ADK.

5

## LO5 · MCP & Sub-Agents

Wrote an MCP server and a hierarchical sub-agent architecture.

NEXT STEPS

# Where to Go Next

1

## Rebuild from memory

Redo the labs without the guide until the patterns are automatic.

2

## Ship something small

Pick one real task at work and build a two-agent system for it.

3

## Publish an MCP server

Wrap a capability your team already uses and share it.

4

## Add evaluation

Write test cases for your agents and measure changes against them.

5

## Watch the ecosystem

Both SDKs move quickly — follow their changelogs.

6

## Keep the human gate

Review agent output before it reaches production.

# Summary

1

## From one agent to many

We moved from a single tool-calling agent to hierarchical multi-agent systems.

2

## Two ecosystems

We built the same architecture twice — OpenAI Agents SDK and Google Gemini ADK.

3

## Vibe coding as method

Engineered context and a written spec — a build method, not a shortcut.

4

## MCP as leverage

A capability written once becomes reusable by every agent you write.

5

## Context isolation

Sub-agents return conclusions, keeping the parent reliable at scale.

6

## Questions?

Ask anything before we move to the assessment.

FINAL ASSESSMENT

# Assessment

## Written (WA)

- Short-Answer Questions (SAQ)
- 50 minutes
- Answer in your own words

## Practical (PP)

- Multi-agent build tasks
- 75 minutes
- Based on the labs you did

## Remember

- Open book — slides and Learner Guide
- Take the Assessment digital attendance (TRAQOM)
- Submit on the LMS: https://lms-tms.tertiaryinfotech.com

ON ASSESSMENT DAY

# Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

# Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- Complete the Certificate and TRAQOM survey on the LMS — both are mandatory.

HAPPY BUILDING

# Thank You!

You can now design, build and deploy collaborative multi-agent AI systems — with vibe coding as your build method.